

Pinpointing Virtual Object Locations in Real World

Tungjai Mady

Computer Game Programming, bachelor's level
2025

Luleå University of Technology
Department of Computer Science, Electrical and Space Engineering

[This page intentionally left blank]

Abstract

This project aims to develop a system to improve video production process, especially for one-person production. The idea is to accurately map out virtual locations in Unreal Engine to real-world locations, using camera tracking with Unreal Engine live link. The solution is to use a prop with reflective materials attached to it, which will act like a detector for detecting actors / props in the virtual world.

Sammanfattning

Detta projekt syftar till att utveckla ett system för att förbättra video produktionsprocessen, särskilt för enmans produktion. Idén är att noggrant kartlägga virtuella platser i Unreal Engine mot verkliga platser, med hjälp av kameraspårning via Unreal Engine Live Link. Lösningen använder ett objekt med reflekterande material, som fungerar som en detektor för att identifiera aktörer och objekt i den virtuella världen.

Table of Content

Introduction.....	1
Background.....	3
Method.....	6
Project Environment.....	6
System Architecture.....	7
Unreal Engine Plugin.....	7
Color Calculation.....	8
Sound Calculation.....	8
System Specifications.....	9
Analysis.....	10
System Latency.....	10
Setup Accuracy Test.....	10
Result.....	12
System Functionality.....	12
Accuracy.....	12
Latency.....	12
Usability for Single-Person Production.....	13
Limitations.....	14
Conclusion.....	15
Reference.....	15

Introduction

The goal was to improve the video production process using mocap, especially for one-person production. A significant challenge faced in video production is aligning the virtual scene created in Unreal Engine^[2] to their corresponding real-world environment.

Motion Capture or (mocap)^[1] for short is a process of recording movement of an object, or people. The recording then will later be used in rendering for film, video games, or animation for realistic characters. This technique works by tracing the motion and then converts it to data, which then will be applied to the virtual model.

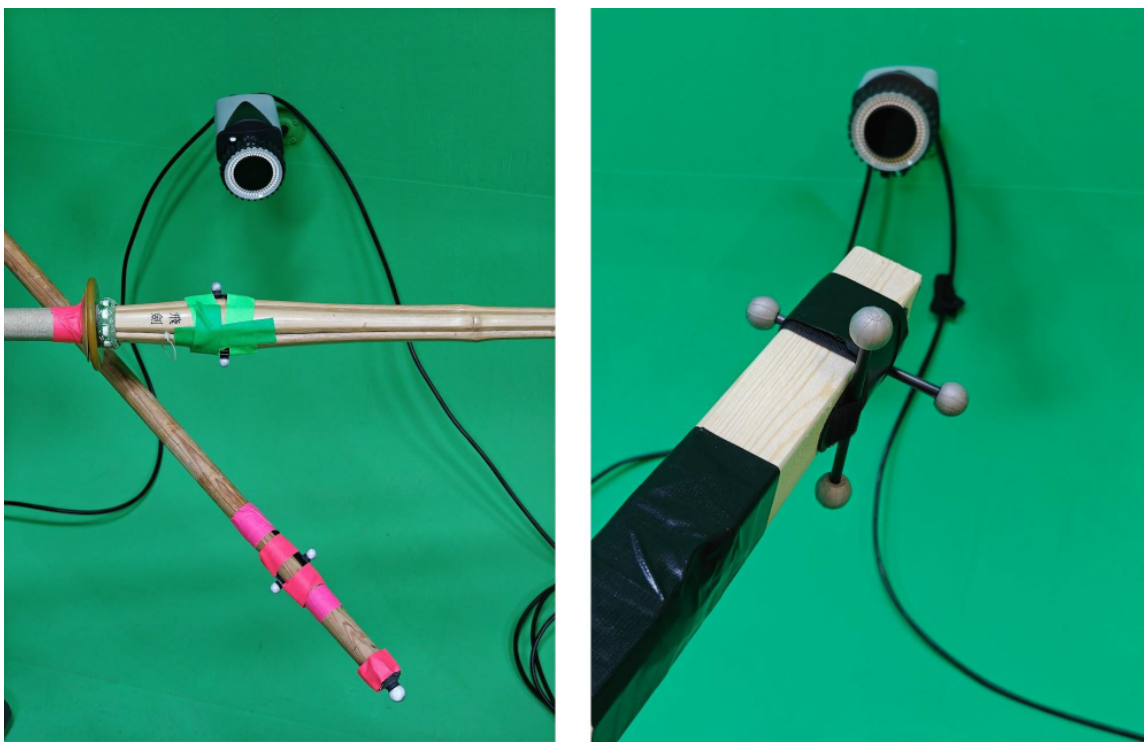


Figure 1.1. Shows a Qualisys motion capture camera with physical props equipped with reflective markers.

The reflective material^[8] is taped around the props, as shown in **Figure 1.1**. This ensures that the reflective material stays in the same place relative to the props. The QTM will then figure out the rigid body, depending on the distance between the reflective material.

To improve the production, the project will focus on precisely mapping virtual locations in Unreal Engine to match the ratio and dimensions of actual real-world settings. Real-time synchronization between the virtual and physical environments will be facilitated through camera tracking technology integrated with Unreal Engine's Live Link plugin,^[5] and Qualysis^[6] providing immediate positional feedback. This integration will greatly enhance the workflow and effectiveness of the production, significantly benefiting if it's an individual.

Background

One of the problems with motion capture in a real-world setting is that the actor or the producer does not have the knowledge of where all the props or objects are in the virtual world. Therefore having a tool to search for the location of the objects from the virtual world will be beneficial, rather than looking at the screen and trying to figure things out.

For instance, if the scene requires an actor to face certain props, but the props are only in the virtual world. The actor will have hard time figuring out where the props are located in the real-world, therefore having a tool that acts as an indicator on where the virtual world object is located in the real-world.

Unreal Engine is a free and powerful real-time 3D creation tool developed by Epic Games.^[7] Mainly known for their game engine due to its high capability, however Unreal Engine is also integrated with a variety of tools for graphic rendering, filmmaking, virtual production and other.

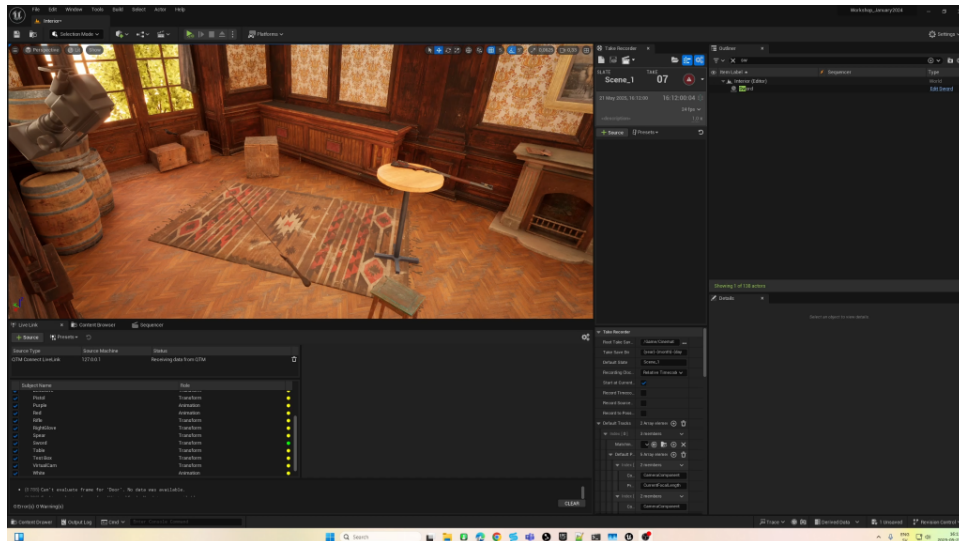


Figure 2.1 Shows a screenshot of the virtual environment created in Unreal Engine. The project, titled *Workshop_January2024*, is from Luleå University of Technology.

An example of what an Unreal Engine scene for motion capturing looks like is shown on **Figure 2.1**. The scene is of an old wooden interior, with props lying around. The position of these props is controlled through QTM via Unreal Engine Live Links.

Live Link^[5] is one of the Unreal Engine plugins. This plugin helps facilitate real-time data streaming from external sources into Unreal Engine. Live Links allows transition of data such as position, rotation, and potentially animation data. In this project, Live Link will act as a primary bridge between the real-world environment and the Unreal Engine virtual world.

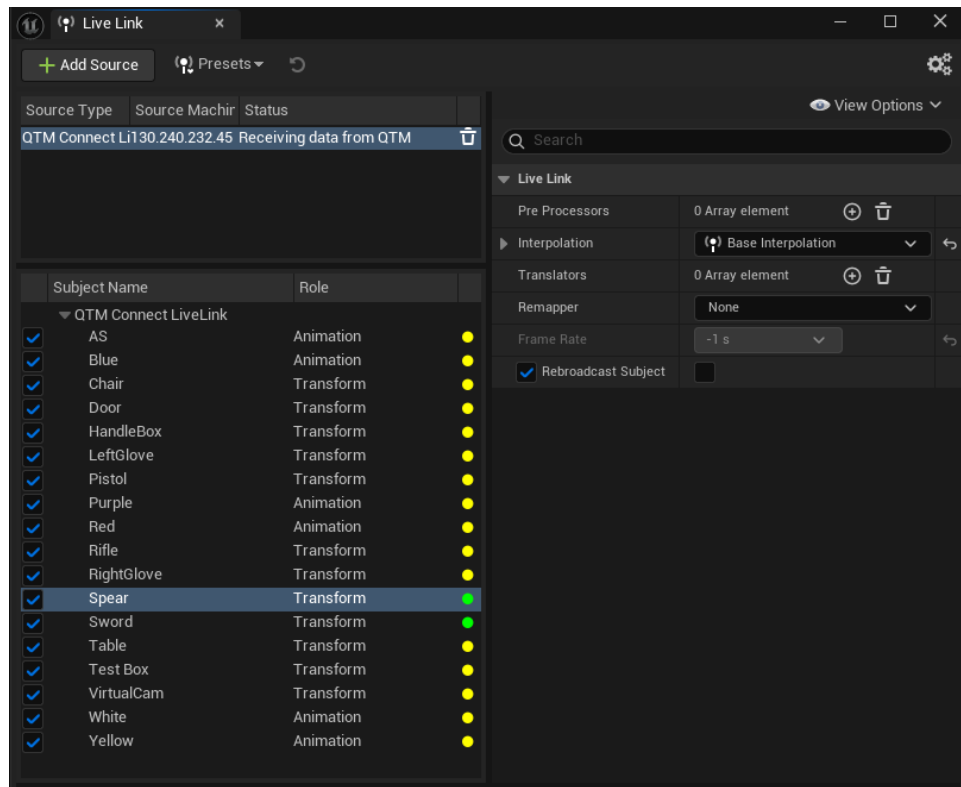


Figure 2.2 Illustrate the Live Link panel in Unreal Engine. The green circle shows active props within the scene, while yellow means that the props is inactive.

Unreal Engine Live Link plugin will receive motion capture data from Qualisys Track Manager (QTM)^[10] at IP address 130.240.232.45 as shown in **Figure 2.2**. The green and yellow status indicators reflect data reception quality for each tracked subject.

Qualisys is a Sweden-based company,^[6] specialized in advanced motion capture technology (mocap), providing a very accurate set of data from physical settings into 3D space. This works by placing cameras around the scene, making sure that the background is consistent, often green background is used for easier separation of subjects. Reflective marker^[8] is separately placed on the objects or human subjects to help with the tracking methods.

Qualisys Cameras^[9] emit infrared light, which will reflect off the reflective marker, and return to the camera's sensors, allowing the computer to compute the necessary data.

The two types of cameras that were used in this project was the Qualisys Miquis^[4], and Qualisys Arqus^[3]. Qualisys Miquis is an affordable motion capture camera used to track motion with low latency and high accuracy, that comes with 3 different models. The camera is also powered by an Ethernet cable, up to 6 cameras in a row before needing to be connected to the power supply. Qualisys Arqus is a high-performing motion capture camera that is highly accurate, and comes with the highest resolution mocap camera in the industry. Another upside of using Arqus is it's durability, it is great for outdoors due to it's being designed being waterproof and dustproof, at the same time able to withstand different temperatures.

Method

Project Environment

Since the purpose of this project is handling qualysis, an unreal engine. It is necessary to set up in the correct environment, the location needs to be equipped with the correct gears and equipment. This setting is located in Studio A156 at Luleå University of Technology, Skellefteå. The room consists of 20+ Qualysis cameras^[9], with a green screen, and all the equipment for the green screen, such as the props for testing, reflective material^[8], and others.



Figure 3.1 Shows the motion capture setup in Studio A156 at Luleå University of Technology.

Qualysis cameras are placed strategically around the studio to be optimized for motion capturing, see **Figure 3.1**. This placement ensures optimal coverage and redundancy, and maximizes tracking accuracy. Each camera will be divided into a group manually, and each group captures data at a time.

System Architecture

The project started with creating a blank C++ Unreal Engine Project and adding Live Link plugins^[5]. This study integrates real-time spatial tracking and digital-physical coordination using Unreal Engine and Qualisys Track Manager (QTM)^[10]. The goal was to create a virtual scene that accurately pinpoints a specific location from the virtual scene within the physical settings.

Qualisys Track Manager (QTM) is the software used to visualize the rigid body created by the user. An additional key feature of QTM is its ability to transmit motion capture data over a network to another computer. This functionality is utilized in this project, as two computers are involved: a personal computer and a university-provided workstation. University's workstation will act as the sender, transmitting the model's position and orientation data via QTM, while the personal computer will act as the receiver via Unreal Engine Live Link.

Creating the plugin in Unreal Engine can be done within Unreal Engine itself^[14], using C++. This plugin will be focused on managing all the objects that will be used for the system such as the detector, and an object that acts as the location. The material of the detector will change its color based on the distance between the detector and the desired location. The detector will turn it as soon as it gets close to the desired location, and turn red when it is far away.

Unreal Engine Plugin

The Unreal Engine plugin was developed directly within Unreal Engine using C++. It acts as the core system for managing real-time proximity detection and giving feedback to the user. The plugin has three main functionalities: taking in an array of actors, providing color-based visual feedback, and generating auditory cues based on distance. In run-time, the plugin receives an array of target actors, each actor contains a Point of Interest (POI) component. This POI component is a predefined transform, usually a scene component attached to the actor's root, and it serves as the reference point for all calculations.

Each frame, the plugin iterates through the array of actors, accessing their respective POI components to determine their world positions. It then calculates to find an actor that has its POI closest to the detector's tip. Then, the distance between the detector tip and its POI is being computed. This distance will be used for both the color interpolation logic and the beeping sounds for auditory feedback. By having the POI as a component, the system allows users to be more flexible and choose specific parts of a rigid body that the user wishes to track or interact with. This will also allow users to be able to introduce new actors or swap between actors. At the same time being able to find multiple points of interest, which make the system flexible and scalable.

Color Calculation

As the detector is close to the point of interest, its mesh changes color based on the proximity: the color of the mesh goes from red when far away to green when close. The color is calculated using linear interpolation. A maximum and minimum distance is predefined where the maximum distance corresponds to the point at which the object is rendered entirely red, and the minimum distance corresponds to full green.

At runtime, the system computes the value between 0.0 and 1.0 using linear interpolation based on the current distance, with 0.0 when it reaches maximum distance or higher, and 1.0 representing the minimum distance or lower. This value will be the intensity of the green channel, while the red channel is inversely proportional to the green channel hence (i.e., $1 - \text{normalizedValue}$). These values will then be combined into a single vector (in the format `(Red, Green, Blue)`), which is the way color is represented in this case, and it will change the color of the detector's mesh.

Sound Calculation

For spatial awareness, the computer uses a proximity-based auditor feedback system. The computer will compute the distance from the detector to the closest point of interest, since there could be more than one point of interest. This logic is executed per-frame within Unreal Engine.

Each frame, the system computes the distance between the detector and the closest point of interest in the scene. The core of the logic involves a conditional check: if the current distance (`mCurrentDistance`) is less than or equal to the predefined maximum audible range (`mMaxDistance`), a normalized value is computed using the built-in function (`FMath::GetRangePct`). This function returns a normalized value between 0.0 and 1.0, based on the current distance, and its defined minimum range and maximum range. A lower distance thus results in a smaller normalized value, hence close proximity.

The normalized value is then passed into a linear interpolation function (`FMath::Lerp`), which adjusts the interval between beeps (`CurrentBeepInterval`) based on proximity. As the user approaches the target, the interval decreases, resulting in more frequent beeping, this will create a sense of closeness for the user.

A timer (`TimeSinceLastBeep`) accumulates elapsed time (`DeltaTime`) per frame. Once the value of this timer exceeds the computed beep interval, a 2D sound (`BeepSound`) is triggered with the built-in function (`UGameplayStatics::PlaySound2D`), and the timer is reset back to 0. If the user exceeds the maximum audible range, the timer is reset without playing any sound, this will make sure no sound is played when the detector is outside the maximum range.

This method will allow users to get proximity auditory based feedback in real time, while the user is interacting within the 3D environments.

Since the majority of the implementation is written in C++, the constructor initializes a root scene component, which serves as the root to all other components attached. Two static

meshes are then created: one representing the detector body, and the other representing the detector tip; the tip also functions as the calculation point. To enable real-time visual feedback, two dynamic material instances are assigned to these meshes, allowing their colors to be updated during runtime with the code. By default, both materials are set to black when the application starts. During each frame, the system continuously evaluates the distance between the detector tip and a predefined target location, adjusting the mesh colors accordingly to reflect this spatial relationship.

System Specifications

Two computing environments were used in the course of this project: a university workstation and a personal desktop computer. Their specifications are detailed below.

Personal Computer:

Operating System: Windows 11 Pro (64-bit)

CPU: AMD Ryzen Threadripper PRO 5945WX (12 cores, 4.09 GHz)

GPU: NVIDIA GeForce RTX 3070 Ti

RAM: 64 GB

University Workstation:

Operating System: Windows 11 Education (64-bit)

CPU: AMD Ryzen 9 7950X (16 cores, 4.50 GHz)

GPU: NVIDIA GeForce RTX 4090

RAM: 96 GB

In this setup, the university workstation acts as the command center for motion capturing. It is directly connected to the Qualisys motion capture system, including all cameras and the Qualisys Track Manager (QTM) software. Therefore, its main job is receiving raw data from all the cameras, processing the images, and then reconstituting the 3D positional data in real time. Once processed, the motion data, such as, position, orientation of those rigid bodies is being transmitted over the Ethernet network.

The personal computer will be the one receiving unit, its running Unreal Engine and the Live Link plugin to visualize and process the incoming data stream. This might seem like an optimization for performance, since the computer can exclusively focus on one task. However, the university workstation is a shared resource, used by multiple individuals and is being reserved for general academic purposes. Therefore, it was not appropriate to host personal project files or custom Unreal Engine configurations on that system. The personal computer was therefore utilized to independently run the Unreal Engine project.

Analysis

System Latency

With this workflow, the latency comes from different components. The camera capture, QTM^[10] processing the image, data transmission to different computers, another computer receiving the data, unreal engine data processing, and unreal visualization. Cameras first capture raw image data, and need to be transferred to the main computer via ethernet cable. According to Qualisys, Miquis cameras, for example, have a latency of approximately 2.9 to 5.5 milliseconds depending on model specifications and hardware performance^[12].

QTM then processes this data, reconstructing 3D positions, typically adding 4 milliseconds depending on the hardware, and complexity of the motion^[11].

The data is then transmitted to a personal computer, via a wired Ethernet connection. The latency introduced by Unreal Engine's rendering pipeline depends on scene complexity, hardware capabilities (GPU performance), and rendering options. Such as, polygon counts, shader complexity, and level-of-detail. This makes the visualization phase a bottleneck in achieving real-time between live motion capture and rigid body. Therefore, balancing visual quality with performance is essential to maintain a smooth and responsive user experience, especially for determining the location of props, which is the goal of this project.

Setup Accuracy Test

The accuracy of the system was evaluated using a physical stick with couples of reflective materials and tracked in real time by Qualisys Track Manager. To perform the test, the stick was manually displaced along a straight path by a known distance of 100 cm, which was measured using a standard ruler by a human being. The position data is then streamed live into the Unreal Engine through the Live Link plugins^[5]. This way it is possible to directly compare between the physical displacement and the displacement in the Unreal Engine.

Three tests were tested out to improve consistency and repeatability, by two different people. For each test, the starting and ending positions of the stick were recorded in Unreal Engine, and the difference was calculated along the Y-axis only, as shown in **Figure 4.1**, as this is the direction that was moved with the physical stick. The measured displacements were: 100.11 cm, 100.13 cm, and 100.29 cm, which makes an average of 100.16 cm. A measurement uncertainty of ± 0.5 cm was assumed due to ruler limitations.

The recorded displacements were as follows:

Start	End	Difference
161.2 cm	61.1 cm	100.1 cm
161.1 cm	61.0 cm	101.1 cm
161.5 cm	61.0 cm	101.1 cm

Table 4.1 Shows measurement data between the start position and end position. The values represent the recorded Y-axis positions of a tracked object in Unreal Engine during a controlled displacement test. The differences reflect the system's measured displacement compared to the intended physical movement of 100 cm.

The observed percentage error, calculated as:

$(\text{Average Measured Value} - \text{Actual Value}) / \text{Actual Value} \times 100$. Resulted in $= (100.16 - 100.0) \times 100 = 0.16\%$. However, it is necessary to take the ruler measurement into consideration which is ± 0.5 cm. The root-sum-square (RSS) method^[13] will combine measured error and the instrumental uncertainty, the combined uncertainty is now:

$$\text{Total Percentage Uncertainty} = (0.16\%)^2 + (0.5\%)^2 = 0.525\%$$

This means that the total percentage uncertainty for this system, based on the testing method used is 0.525%. The results mean that the system has a high level of accuracy for motion capturing.



Figure 4.1 Shows the physical and virtual measurement during the accuracy test. The top portion of the image shows the displacement of the tracked object in Unreal Engine, while the bottom portion displays the corresponding physical setup, with a stick marked and measured using a standard ruler across a 100 cm range.

Result

System Functionality

The system successfully pinpoints the location the real-world positions relative to their virtual locations in Unreal Engine. The detector object uses proximity between its location and a predefined virtual target location. The detector object will change its color interpolating from red (far) to green (near) in real-time. This visual feedback provides sound cues, allowing users to move around and align themselves or physical objects with the virtual scene, even without direct screen monitoring.

Accuracy

The result of the 100 cm physical displacement test gives a result of 100.11 cm, 100.13 cm, and 100.29 cm—averaging 100.16 cm. The resulting systematic deviation of 0.16 cm corresponds to an observed percentage error of 0.16%. When combined via the root-sum-square method with the ± 0.5 cm ruler uncertainty (instrumental error), the overall percentage uncertainty is 0.525%. This shows that the system has a sub 1% error, which shows the system is more than capable of pinpointing any target of interest in real time.

Latency

The system latency in this workflow comes from different stages: image capture, data processing, network transmission, and visualization. Initially, there's a few milliseconds during the image capturing stage from the Qualisys Miquis cameras^[12], the actual latency will also depend on the camera model, and the computer hardware. The Qualysis Task Manager^[10] will also take a couple of milliseconds to construct 3D position, however this is also dependent on the hardware and motion complexity.

At the same time, the data is being transmitted over Ethernet to a separate machine running Unreal Engine via Live Link^[5]. Latency during this stage is largely dependent on scene complexity, rendering settings, and GPU capabilities. This also depends on the Unreal Engine's setting, such as, polygon count, shader usage, and level of detail. This means that finding the balance between visual appealing, and responsiveness is critical to ensuring real-time performance.

Usability for Single-Person Production

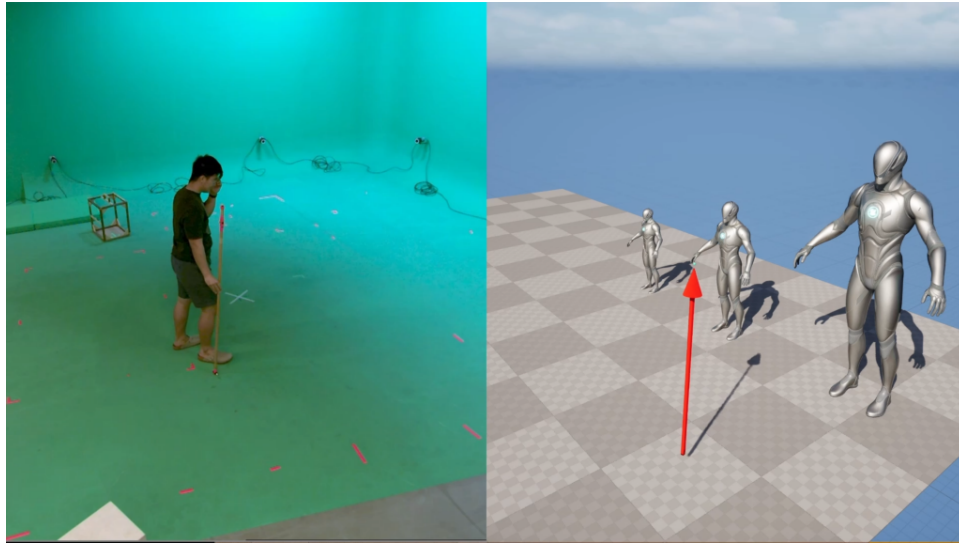


Figure 5.1 illustrates that the detector turns red, meaning it indicates the point of interest is nowhere near the detector, and there won't be any beeping sounds.

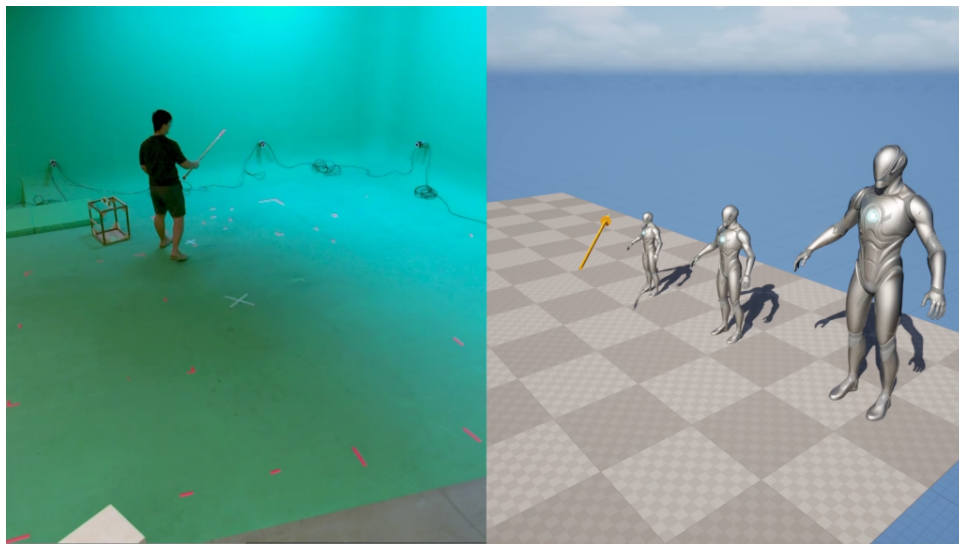


Figure 5.2 shows the detector turning yellowish means it is relatively close to the point of interest. The beep sound will depend on the distance of the detector to the point of interest.

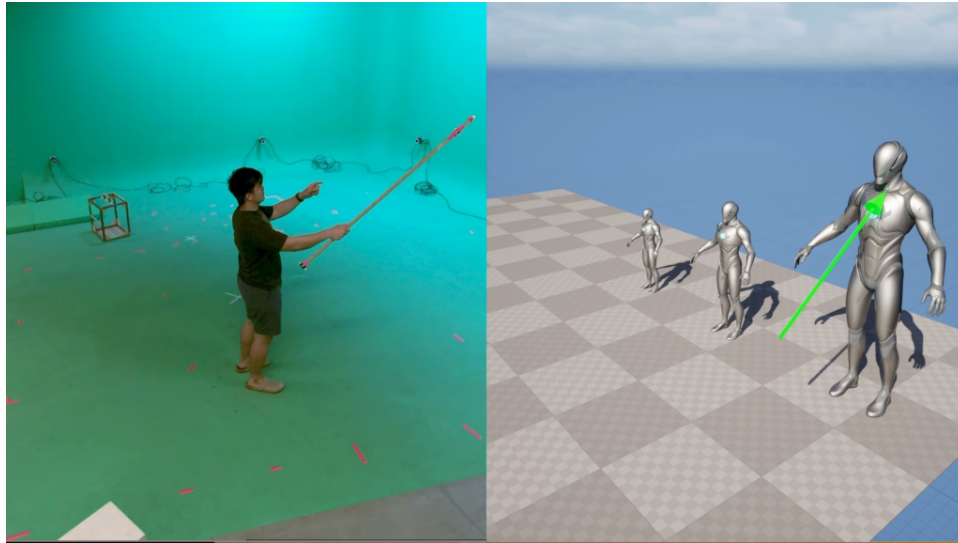


Figure 5.3 shows when the detector turns green, which means it is at a point of interest, and the sound is beeping in high frequency.

Limitations and Future Work

One of the limitations within this project is the decision to make a dedicated actor class (`AMainObject`), this is the class that the detector will take in as an input. While its approach works perfectly fine, as long as the user derived from this class, it reduced the reusability across different projects.

The system requires that all searchable targets be derived from (`AMainObject`) class, since the detector requires a specific world space location, which is a method form (`AMainObject`) class. This design creates a constraint, especially for projects that the actor did not inherit from this class, which cannot be detected by the detector. Therefore tedious work needs to be done in order to convert the actor from its class, to inherit from the (`AMainObject`) class, or else the plugin is essentially useless.

On the other hand, a more flexible design would have been to implement the detection logic as an actor component instead. Components in Unreal Engine are flexible, and can be attached to any actor despite their class inheritance. An ideal way to do it is to make a custom `UActorComponent`, which will eliminate all the restriction that only actors need to be derived from a specific base class.

Mechanic wise, one of the ideas is to have a reverse detector. Instead of having a detector and going around the scene, the stick will spawn a small actor at the tip periodically. The overall idea is to be able to map out the scene in real-life out in Unreal Engine. This will help the user to identify their bounding volume.

Using the detector means that the user will have to manually use it, hence not having the ability to be hand-free. The other solution is the use of a drone. The goal is to use the drone to fly to the point of interest, instead of moving the stick around and trying to find the exact location.

This is doable since there's an open source Qualisys Drone SDK available online, it works well with qualisys. Drone will be attached with a couple of the reflective materials, QTM will be able to detect the location of the drone, and send that data to the Unreal Engine. Unreal Engine will then compute the instruction for the drone, and finally call the Drone SDK to execute the instruction to the drone itself. One of the benefits here is the user will be able to find the point of interest hand-free, at the same time the drone can stay at the point of interest as long as there's a battery.

At the moment, the detector only considers the location of the point of interest, not the direction. One of the considerations is to include the ability to find the direction as well, this can be done by adding the direction vector on the AMainActor. The detector itself has a direction to it, therefore it won't be difficult to do a dot product, and find if the two vectors are facing the same direction or not. Some other cues could be added when the direction aligns, or the user can manually select a location finder mode or rotation finder mode. This could be useful if the user wanted to find the direction and rotation of the light source in physical-settings.

Conclusion

The primary objective of this project was to improve the workflow of single-person video production by creating a system capable of accurately mapping virtual object positions in Unreal Engine^[7] to their corresponding locations in the physical world. This was achieved through the integration of Qualisys^[6] motion capture technology and Unreal Engine's Live Link plugin^[5], this allows data to be streamed between the two environments, and compute a proper feedback for the user.

The system successfully provides immediate feedback to the user, using a detector object that will respond to the object of interest in real-time by visual and auditory cues. Visual cues are provided through color interpolation from red (far away) to green (relatively close), while auditory cues are provided through beeping frequency based on distance, low to no frequency (far away), high frequency (relatively close). Together, these features allow real-time response for the user, without extra help or screen monitoring, improving single-person video production.

Accuracy test confirmed that the system is capable of high precision, with a measured average error of just 0.16 cm (when tested with 100 cm displacement) and a combined uncertainty of 0.525%. Latency analysis shows that the system's responsiveness is mainly due to the camera capture rates, network transmission, and Unreal Engine's rendering performance, all of which remained within acceptable real-time margins for interactive production.

However, design limitations were identified. Specifically, the use of a dedicated actor class (`AMainObject`) reduced flexibility for the existing actors, or actors that do not inherit the dedicated class. A component-based approach would have allowed greater flexibility and usability without requiring inheritance constraints.

In summary, this project demonstrates a practical, accurate, and scalable solution for enhancing virtual production in small-scale or solo creator environments. It bridges the gap

between virtual and real-world alignment. It is also a solid foundation that can be improved in the future, such as, changing it to be a component-based extension, more complex scenes, or other.

Reference

- [1] [*Mocap*](#), “The Science and Art Behind Motion Capture”. Accessed: 6 May 2025
Available at: <https://www.studiobinder.com/blog/what-is-mocap-definition/>
- [2] [*UnrealEngine*](#), “Official Documentation for Unreal Engine”. Accessed: 6 May 2025
Available at: <https://www.unrealengine.com/en-US>
- [3] [*QualisysArqus*](#), “Qualisys High Performance Camera”. Accessed: 6 May 2025
Available at: <https://www.qualisys.com/cameras/arqus/>
- [4] [*QualisysMiquis*](#), “Qualisys Camera”. Accessed: 6 May 2025
Available at:
https://www.qualisys.com/cameras/miquis/?gad_source=1&gbraid=0AAAAADbwGw1IOxWmrwyNbse3u23z7PmKE
- [5] [*LiveLink*](#), “Live Link Plugin Integration for Unreal Engine”. Accessed: 6 May 2025
Available at:
<https://dev.epicgames.com/documentation/en-us/unreal-engine/live-link-in-unreal-engine>
- [6] [*Qualisys*](#), “Official Website of Qualisys”. Accessed: 6 May 2025
Available at: <https://www.qualisys.com/>
- [7] [*EpicGames*](#), “Developer of Unreal Engine”. Accessed: 6 May 2025
Available at: <https://www.epicgames.com/site/en-US/home>
- [8] [*TrackingMarker*](#), “Reflective Tracking Markers”. Accessed: 6 May 2025
Available at: <https://www.qualisys.com/accessories/markers/>
- [9] [*QualisysCamers*](#), “List of Qualysis Cameras”. Accessed: 6 May 2025
Available at: <https://www.qualisys.com/cameras/>
- [10] [*QTM*](#), “Qualisys Track Manager - Software Documentation”. Accessed: 6 May 2025
Available at:
https://www.qualisys.com/software/qualisys-track-manager/?gad_source=1&gbraid=0AAAAADbwGw1J0zljELC9bJ_oN649iHBQg
- [11] [*QTM*](#)[*Latency*](#), “Qualisys Track Manager (QTM)”. Accessed 13 May 2025
Available at: https://cdn-content.qualisys.com/2019/10/PI_Qualisys_Track_Manager.pdf
- [12] [*Miquis*](#)[*Latency*](#), “Introducing Miquis M5”. Accessed 13 May 2025
Available at: <https://www.qualisys.com/campaign/introducing-miquis-m5/>

[13] [RSS](#), “Root Sum Square” . Accessed 21 May 2025

Available at:

<https://www.fiveflute.com/guide/introduction-to-root-sum-squared-rss-tolerance-analysis/>

[14] [UnrealEnginePlugin](#), “Plugins”. Accessed 27, May 2025

Available at:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/plugins-in-unreal-engine>

[15] [UnrealEngineFMath](#), “FMath. Accessed 27, May 2025

Available at:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Runtime/Core/Math/FMath>